

format string 0

[PicoCTF '24] format string 0

This challenge is a fairly simple binary exploitation challenge that relies on a buffer overflow to cause a segmentation fault.

Distribution

This challenge is nice enough to provide both the binary file `format-string-0` and the source code in `format-string-0.c`. file and `checksec` reveals that:

```
Ant32-picoctf@webshell:~$ file format-string-0
format-string-0: ELF 64-bit LSB executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=73480d84a806aebddd86602609fcab2052c8fa13, for GNU/Linux 3.2.0,
not stripped

Ant32-picoctf@webshell:~$ checksec format-string-0
[*] '/home/Ant32-picoctf/format-string-0'
Arch:          amd64-64-little
RELRO:         Partial RELRO
Stack:         No canary found
NX:            NX enabled
PIE:           No PIE (0x400000)
SHSTK:         Enabled
IBT:           Enabled
Stripped:      No
```

Based off this we can see the binary is relatively unprotected.

Solution

I used `gdb` for dynamic analysis of the binary and `objdump -M intel` for static analysis of the disassembly.

Jumping straight into `main()` the first thing we see is the typical Pico flag setup where the process will try to open a file containing the flag. If the file is unable to be located, it prints an error message to the screen and exits. The next and perhaps most important thing is a `signal(SIGSEGV, sigsegv_handler)`. Clearly this program is prepared to catch a segmentation fault, however the second parameter is not `SIG_IGN` or `SIG_DFL` -- it is a custom signal handler: `sigsegv_handler()`. Sure enough the first function defined in the

source is this mysterious `sigsegv_handler()` . Here we can see the program handles a segmentation fault by... printing the flag!

```
void sigsegv_handler(int sig) {  
    printf("\n%s\n", flag);  
    fflush(stdout);  
    exit(1);  
}
```

Okay... so this seems simple enough. To get the flag we need to cause a segmentation fault in the process. Well the next thing of note in `main()` is a call to `serve_patrick()` -- a function which may serve us in this goal. The first local variable created by this function is the 32 character array, `choice1` . This variable is filled in the following line with `scanf("%s", choice1)` . These two lines are all we need to create segmentation fault in our process. The number of bytes to overflow by is not an exact number we need to calculate, however from my testing, it is greater than 40, which is already greater than the `BUFSIZE` of 32. The number I chose came from a few lines down:

```
if(count > 2 * BUFSIZE) .
```

`2 * BUFSIZE = 64`, so +1 is 65. You can pipe (if running the program on the remote host like you are supposed to) `python3 -c "print('a'*65)"` into `netcat` to feed the program our payload that will cause the `SIGSEGV` . After that you should receive the flag!